

Ejercicio 1 – Transformaciones

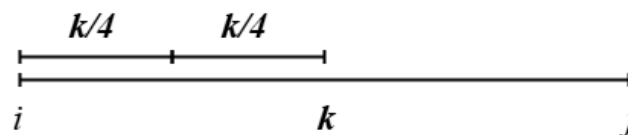
Se dispone de N monedas (siendo N potencia de 2), de las cuales se sabe que una de ellas es falsa, lo que se manifiesta en que su peso es distinto a las demás (no se sabe si mayor o menor). Para poder determinar cuál es la moneda falsa, se cuenta con un objeto Java `monedas` de la clase `Monedas` que identifica cada una con un índice en el intervalo $[0, N)$. Para la resolución del problema, dicho objeto permite la llamada al método siguiente:

```
int pesar (int i1, int j1, int i2, int j2)
```

El método “`pesar`” realiza una comparación del peso total de las monedas en el intervalo $[i1, j1)$ con el de las monedas en el intervalo $[i2, j2)$. El valor devuelto por el método será un entero negativo, cero o un entero positivo según si el peso del primer intervalo es menor, igual o mayor que el del segundo intervalo, respectivamente.

Siguiendo el esquema *Divide y Vencerás*, se desea determinar una estrategia para encontrar la moneda falsa realizando el menor número de llamadas posibles al método “`pesar`” en el caso peor. Para ello, se definirá un algoritmo recursivo que, dado un conjunto de k monedas, selecciona dos subgrupos de $k/4$ monedas con índices consecutivos y realiza una comparación entre ellos. Según el resultado, se distinguen dos casos distintos:

- Si el peso de ambos es el mismo, significa que la moneda falsa no se encuentra en los subgrupos escogidos. En este caso, se realizaría la búsqueda en el grupo de monedas restantes.
- Si la moneda falsa se encuentra en alguno de los subgrupos escogidos, la búsqueda se realizaría en el conjunto de los subgrupos que se pesaron.



El algoritmo iterará hasta localizar la moneda falsa y devolverá el índice correspondiente a la misma. El método que lo implementa tiene la siguiente cabecera:

```
int monedaFalsa (Monedas monedas)
```

Supóngase definido e implementado el método privado

```
int monedaFalsaSi3 (int i, int j)
```

que devuelve el índice correspondiente a la moneda falsa si $j-i \leq 3$.

SE PIDE:

- Defina la función recursiva que representa el algoritmo explicado anteriormente.
- Desarrollar el algoritmo descrito en su versión recursiva.
- Obtener la versión iterativa imperativa del algoritmo del anterior apartado.

Tiempo estimado: 45 minutos – Puntuación: ADDA 25% - EDA 30%

SOLUCIONES:

A) Definición de la función recursiva:

$$mf(i, j) = \begin{cases} monedaFalsaSi3(i, j), & j - i \leq 3 \\ mf\left(i, i + 2\left\lfloor \frac{j-i}{4} \right\rfloor\right), & j - i > 3, \text{pesar}\left(i, i + \left\lfloor \frac{j-i}{4} \right\rfloor, i + \left\lfloor \frac{j-i}{4} \right\rfloor, i + 2\left\lfloor \frac{j-i}{4} \right\rfloor\right) \neq 0 \\ mf\left(i + 2\left\lfloor \frac{j-i}{4} \right\rfloor, j\right), & j - i > 3, \text{pesar}\left(i, i + \left\lfloor \frac{j-i}{4} \right\rfloor, i + \left\lfloor \frac{j-i}{4} \right\rfloor, i + 2\left\lfloor \frac{j-i}{4} \right\rfloor\right) = 0 \end{cases}$$

B) Implementación del algoritmo en su versión recursiva:

```
public int monedaFalsa(Monedas monedas) {
    return monedaFalsaRecursivo(0, monedas.size(), monedas);
}

private int monedaFalsaRecursivo(int i, int j, Monedas monedas) {
    int indiceMonedaFalsa = -1;
    if (j-i <= 3) {
        indiceMonedaFalsa = monedaFalsaSi3(i, j);
    } else {
        if (monedas.pesar(i, i+(j-i)/4, i+(j-i)/4, i+2*((j-i)/4)) != 0) {
            indiceMonedaFalsa = monedaFalsaRecursivo(i, i+2*((j-i)/4), monedas);
        } else {
            indiceMonedaFalsa = monedaFalsaRecursivo(i+2*((j-i)/4), j, monedas);
        }
    }
    return indiceMonedaFalsa;
}
```

C) Implementación del algoritmo en su versión iterativa imperativa:

```
public int monedaFalsalterativa(Monedas monedas) {
    int i = 0;
    int j = monedas.size();

    while (j - i > 3) {
        if (monedas.pesar(i, i+(j-i)/4, i+(j-i)/4, i+2*((j-i)/4)) != 0) {
            j = i+2*((j-i)/4);
        } else {
            i = i+2*((j-i)/4);
        }
    }
    return monedaFalsaSi3(i, j);
}
```